

Implementation of WebRTC in Real-time Communication Applications

Anamarija Papić

University of Split, University Department of Professional Studies, Split, Croatia
anamarija.papic@oss.unist.hr

Tea Bašić

University of Split, University Department of Professional Studies, Split, Croatia
tea.basic@oss.unist.hr

Nikola Grgić

University of Split, University Department of Professional Studies, Split, Croatia
nikola.grgic@oss.unist.hr

Abstract. This paper describes WebRTC (Web Real-time Communication) as a collection of standardized open-source technologies that enable direct peer-to-peer real-time communication over the internet, both through modern web browsers and native clients in the case of mobile applications. End users can exchange audio, video, and data without the need for additional plugins or dedicated software. WebRTC consists of the WebRTC protocol and the WebRTC API specified for JavaScript, while a library providing the same functionality is available for native clients. Software engineers can integrate WebRTC into modern application architectures in order to deliver robust, low-latency solutions with built-in security, since encryption is mandatory. WebRTC can be roughly broken down into four sequential steps: signaling, connection establishment, securing the connection, and the actual communication, where each subsequent step only begins once the previous one has been completed successfully. In the first phase, signaling, an out-of-band channel (e.g. REST or WebSocket) is used by peers for exchanging SDP messages containing information about media streams, codecs, and network parameters, and to negotiate session parameters. The next step is connecting, where ICE and NAT/firewall traversal mechanisms, together with STUN/TURN infrastructure, are used to discover a viable network path between participants. Once the connection has been established, the transport layer is secured using DTLS and SRTP, which provide encryption and integrity for media and data streams. Finally, the actual communication takes place over RTP for audio and video and SCTP for data channels, with these protocols allowing the application to tune latency, packet loss handling, and reliability according to its requirements. These steps will be described in detail in the paper. The paper will also introduce various parts of the WebRTC API, including interfaces for working with media devices and for establishing peer-to-peer connectivity between clients, with emphasis on the most important interfaces, dictionaries, and types used to set up, open, and manage WebRTC connections. Finally, the focus will be on a practical integration example, presenting the design and implementation of a custom WebRTC-based solution for in-browser audio and video communication between multiple participants, using a modern web application that demonstrates the application of the described concepts and the use of the WebRTC API.

Key words: *audio/video streaming, NAT traversal, peer-to-peer, real-time communication, WebRTC*

1. Introduction

WebRTC technology is a collection of standardized open-source (FOSS) technologies that enable direct peer-to-peer real-time communication over the internet, through modern web browsers or native clients in the case of mobile applications. End-users can exchange audio, video, and data without the need for plugins or special software. WebRTC consists of

the WebRTC protocol and the JavaScript-specific WebRTC API, while a library providing the same functionalities is available for native clients. The source code repository for the WebRTC project is available at <https://webrtc.googlesource.com/src>.

The WebRTC protocol is essentially composed of a series of standardized protocols and is therefore often described as a collection of protocols because it orchestrates various protocols and mechanisms for establishing, maintaining, and securing real-time communication, as shown in Figure 1 [1].

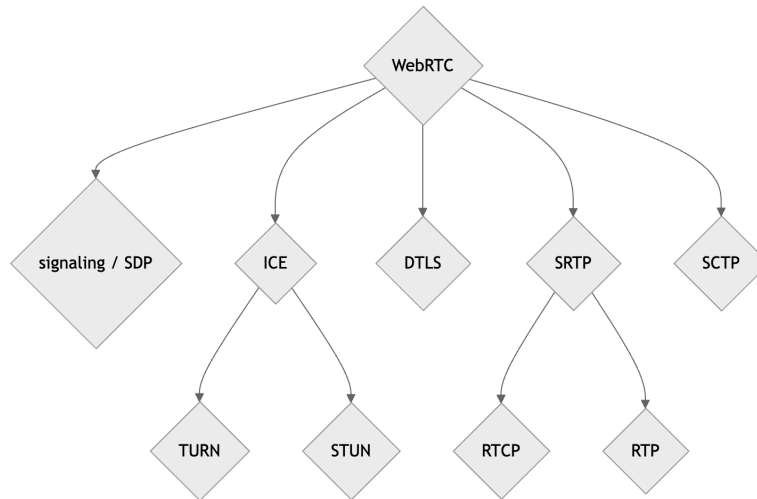


Figure 1 WebRTC as a collection of protocols

It emerged as an attempt to integrate real-time speech and video technologies, which were previously closed and proprietary, directly into web browsers as an open standard. Google acquired the company Global IP Solutions (GIPS) in 2010 and released the code for their RTC components under the open BSD license, which was a key step in the development of WebRTC. Subsequently, Google launched the WebRTC project in 2011, and standardization took several years, requiring significant efforts and collaboration between two standardization bodies: W3C (World Wide Web Consortium) and IETF (Internet Engineering Task Force), as well as a large number of companies and individuals from the industry [14]. In addition to Google, companies such as Mozilla, Microsoft, Apple, Intel Corporation, Cisco, and Ericsson participated in the development of specifications and implementation, and support was gradually introduced in all modern web browsers, making WebRTC the standard mechanism for in-browser audio-video communication.

Today, it is applied in a wide range of applications, from video conferencing to online games, remote control, IoT (Internet of Things) communication, and other scenarios that require low latency and high transmission quality.

2. WebRTC Network Topology

WebRTC topologies can be broadly classified into two categories: peer-to-peer (P2P) topologies and client/server topologies.

In the simplest case, two endpoints establish a direct peer-to-peer connection and exchange media streams and data. For group scenarios, multiple different topologies can be used, depending on the number of participants, available bandwidth, and application requirements.

A full mesh topology implies that each participant is directly connected to every other participant. Each client must then separately encode and send its video stream to each remote participant, which increases network load and limits the practical number of participants, but offers relatively simple implementation without additional server infrastructure.

Alternatively, a hybrid mesh topology reduces the number of direct connections by forwarding data through individual participants.

Client/server-type topologies use specialized servers: SFU (Selective Forwarding Unit), which selectively forwards incoming streams to all clients, and MCU (Multipoint Conferencing Unit), which merges multiple input streams into a single composite output stream [15].

Figures 2 – 6 illustrate various WebRTC topologies.

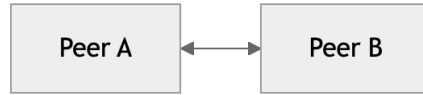


Figure 2 Peer-to-peer topology

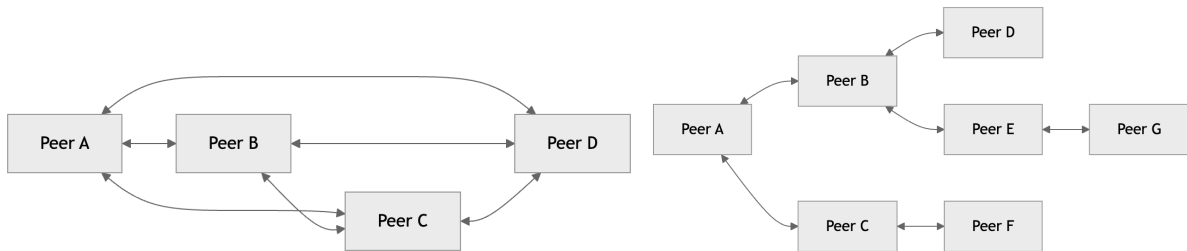


Figure 3 Full mesh topology

Figure 4 Hybrid mesh topology

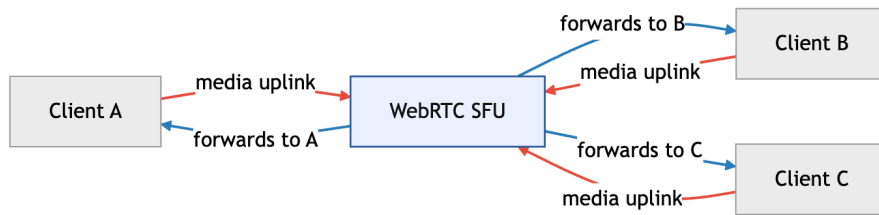


Figure 5 SFU topology

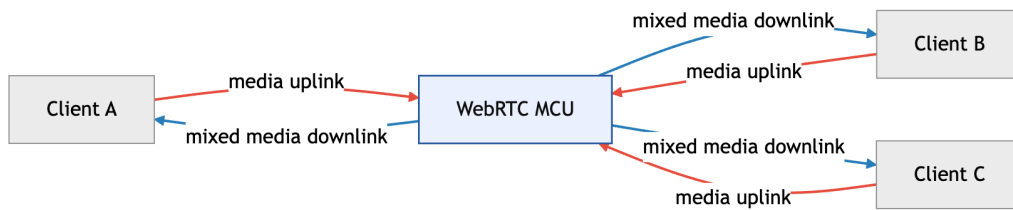


Figure 6 MCU topology

The application described in this paper uses a full mesh topology, meaning that each client in the room establishes one `RTCPeerConnection` link to every other participant.

3. Phases of WebRTC Connection Establishment

The establishment of a WebRTC connection can be conceptually broken down into four sequential steps: signaling, connection setup, connection securing, and the actual communication itself, where each subsequent step begins only after the previous one has been fully and successfully completed. The first phase, signaling, encompasses all steps by which two (or more) participants “find” each other and agree on how they will communicate, before the actual audio-video data transmission begins.

Signaling is not standardized in WebRTC itself but occurs out-of-band and is considered part of the application logic — the development team freely chooses protocols and

message formats (e.g., REST, WebSocket, etc.). First, participants establish a connection to a common signaling channel and somehow identify the room or conversation they are participating in. Then, through that channel, they exchange SDP (Session Description Protocol) messages [4] containing information about which media streams they want to send and which codecs and configurations they support, as well as the corresponding signaling messages (offer/answer, candidates, etc.) needed to establish the connection. Signaling is illustrated in Figure 7.

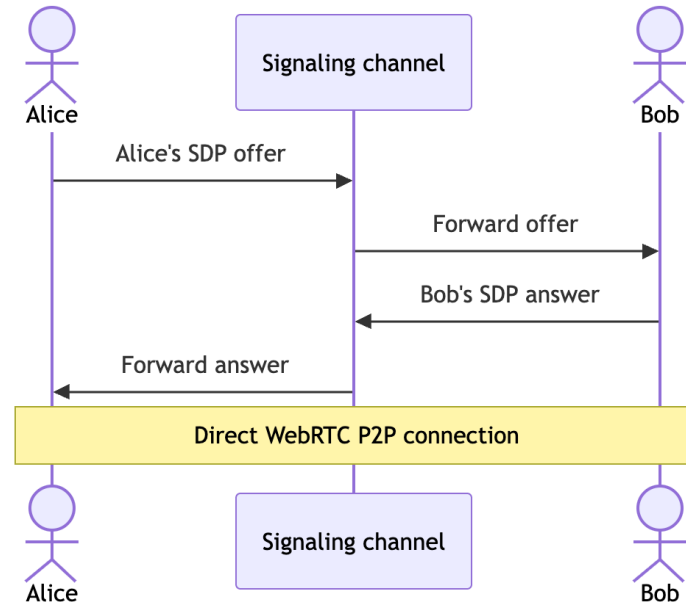


Figure 7 Signaling

After the basic session configuration has been exchanged via signaling, the second phase follows — connection setup, in which WebRTC attempts to establish a feasible network path between participants. In real-world conditions, most clients do not have a public IP address but are located behind routers, NAT (Network Address Translation) devices, and firewalls, which prevents direct addressing between two endpoints. To achieve peer-to-peer communication despite this, WebRTC uses a combination of the ICE (Interactive Connectivity Establishment) [9] framework and STUN (Session Traversal Utilities for NAT) [10] and TURN (Traversal Using Relays around NAT) [5] infrastructure for so-called NAT traversal, i.e., finding paths through the network configuration.

ICE gathers various network “candidates” on each side — so-called ICE candidates — which include local addresses, public addresses obtained via STUN servers, and, if necessary, relay addresses obtained via TURN servers. These candidates are then exchanged via the signaling channel and systematically tested to find a pair of candidates through which bidirectional communication can be established with the lowest possible latency. Testing of candidate pairs (connectivity checks) is shown in Figure 8. The figure shows two ICE agents, each with three different candidates (resulting in a total of nine candidate pairs). In the ideal case, a direct P2P path is selected (e.g., via a public address discovered by STUN), and if that is not possible due to NAT or firewall restrictions, communication is redirected via a TURN server that acts as a relay between participants. In this way, the connection setup phase dynamically adapts the network path to ensure that media streams can be transmitted even in complex network environments, always preferring the least “expensive” paths.

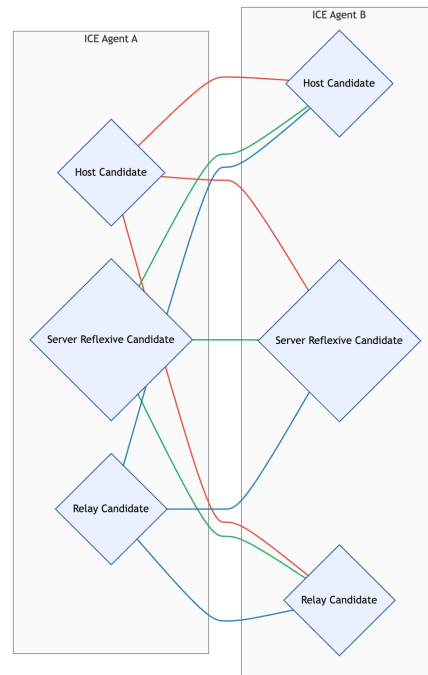


Figure 8 ICE candidates and pair testing

In the third phase, connection securing, WebRTC ensures that all media and data streams are transmitted securely. For this purpose, it uses a combination of two protocols: DTLS (Datagram Transport Layer Security) [7] and SRTP (Secure Real-time Transport Protocol) [3]. DTLS is the UDP version of TLS used in HTTPS, responsible for securely negotiating cryptographic parameters and exchanging keys over unreliable UDP, thereby protecting against eavesdropping, message forgery, and their modification during transmission. SRTP extends RTP by adding encryption of media packet contents, integrity checking, and protection against replay attacks, while remaining lightweight enough to meet real-time communication requirements. In WebRTC, the keys used in SRTP are not sent as plain text but are derived from the DTLS session [2][8] in a process known as DTLS-SRTP — first, a secure DTLS channel is established between participants, then the necessary key material for SRTP is generated from it, and after that, audio and video streams are encrypted and transmitted via SRTP. This makes WebRTC require all media traffic to be encrypted and authenticated, with no possibility of disabling encryption in production implementations, thereby ensuring end-users a high level of communication confidentiality and protection.

In the fourth and final phase, actual communication takes place. WebRTC uses two protocols for actual data exchange between participants: RTP [11] and SCTP [12]. RTP is responsible for transmitting audio and video streams in real time: packets with media frames carry timestamps and sequence numbers, enabling synchronization, packet loss detection, and adaptation of playback to network conditions. In WebRTC, RTP is almost always used in combination with SRTP, which adds encryption and integrity protection, so actual media traffic is transmitted via SRTP packets over the previously established secure channel. For arbitrary data streams, WebRTC uses SCTP (Stream Control Transmission Protocol) encapsulated within DTLS, enabling reliable, order-guaranteed data exchange between participants, with logical division of traffic into multiple independent streams.

4. Practical Implementation

This section presents WebRTC implementation within the tutoring web application developed in [6] that connects students and tutors, facilitating the organization and management of online and onsite tutoring sessions. The application enables students to easily

find and book tutoring sessions, while tutors can efficiently manage their own schedules, offers, and bookings. Participation in online tutoring sessions is supported by integration of peer-to-peer WebRTC technology for video calls. The full-stack web application was developed in the React development framework Next.js, while a relational PostgreSQL database managed via Prisma ORM and hosted on the Supabase platform is used for data storage.

Since WebRTC does not specify the signaling method, this application uses the real-time functionalities of the Supabase platform as the signaling channel. This means that the messages needed to establish the connection are exchanged via the Supabase real-time channel, while the media streams themselves (audio and video) are established directly between participants' browsers after agreeing on the parameters. In this way, the data layer (Supabase/PostgreSQL) and the communication layer (WebRTC) are integrated into a whole — Supabase ensures reliable and scalable exchange of signaling messages, while WebRTC handles real-time multimedia transmission.

Figure 9 shows the architecture diagram of the implemented application, which aims to illustrate the key components and layers of the system and their interactions and communication.

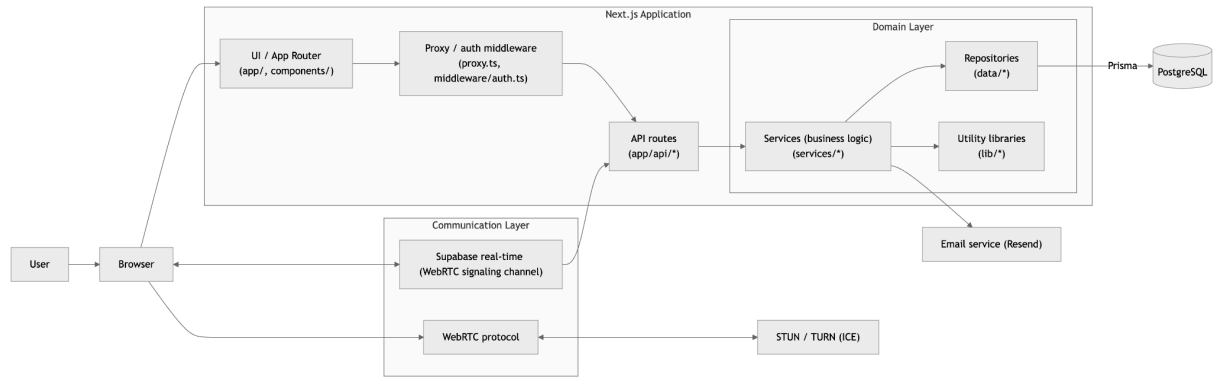


Figure 9 Application architecture diagram

Through the LessonMode enumeration, which can have values ONLINE and ONSITE, support for online tutoring is modeled in the application domain. This way, online and live sessions are clearly distinguished in the business logic and user interface.

Online tutoring sessions take place through WebRTC video calls in so-called meeting rooms. Room initialization is implemented in the client-side React component RoomCall, which manages the signaling client, local media streams, and the map of WebRTC connections to other participants.

To enable group online tutoring sessions, video calls are implemented in a full mesh topology, where each participant establishes a separate RTCPeerConnection link to every other participant.

Supabase channels are used for exchanging signaling messages; above them is defined a module with message schemas (e.g., OfferMessage, AnswerMessage, CandidateMessage) that handles serialization and validation. The WebRTC session lifecycle follows the standard pattern: the initiator creates an offer [13] (createOffer), sets it as the local description (setLocalDescription), and sends it to the other party via signaling. The recipient sets the offer as the remote description (setRemoteDescription), creates an answer (createAnswer), sets it locally, and returns it to the initiator, while ICE candidates are exchanged in parallel until a stable connection is established. After the connection is established, local audio and video streams are added to all relevant connections, and the RoomCall component displays incoming remote streams in the interface. At call termination,

a cleanup procedure is implemented: stopping local `MediaStreamTrack` objects, removing remote streams, and closing all `RTCPeerConnection` instances.

4.1 ICE Server Configuration and Peer Connection Creation

WebRTC peer connections are created with ICE servers configured for NAT traversal using STUN and TURN, as shown in Listing 1:

```
1 const ICE_SERVERS: RTCIceServer[] = [  
2   { urls: ["stun:stun.l.google.com:19302"] }  
3 ]  
4  
5 const pc = new RTCPeerConnection({ iceServers: ICE_SERVERS })
```

Listing 1 Peer connection initialization

The `RTCPeerConnection` constructor accepts an `iceServers` configuration containing STUN server addresses for candidate gathering. In this implementation, Google's public STUN server is used as a fallback, though production deployments typically include private TURN servers for improved reliability behind restrictive firewalls.

4.2 Local Media Stream Acquisition

Before establishing peer connections, local audio and video streams must be obtained from the user's media devices, as illustrated in Listing 2:

```
1 const constraints: MediaStreamConstraints = {  
2   audio: true,  
3   video: true  
4 }  
5  
6 const stream = await navigator.mediaDevices.getUserMedia(constraints)  
7  
8 for (const track of stream.getTracks()) {  
9   pc.addTrack(track, stream)  
10 }
```

Listing 2 Acquiring local media

The `getUserMedia()` call requests access to audio and video devices. The returned `MediaStream` contains `MediaStreamTrack` objects for each active device. These tracks are then added to the `RTCPeerConnection` via `addTrack()`, making them available for transmission to remote peers via RTP.

4.3 Offer/Answer Negotiation

The initiating peer creates an SDP offer describing its media capabilities and local candidates, as shown in Listing 3:

```
1 const offer = await pc.createOffer()  
2 await pc.setLocalDescription(offer)  
3  
4 await signalingChannel.send({  
5   type: "offer",  
6   from: userId,  
7   to: peerId,  
8   sdp: offer.sdp  
9 })
```

Listing 3 Creating and sending an offer

The `createOffer()` method generates an `RTCSessionDescription` containing an SDP string that describes the offer. This is set as the local description via `setLocalDescription()`, then transmitted to the peer through the out-of-band signaling channel.

The receiving peer processes the offer, creates an answer, and returns it, as depicted in Listing 4:

```
1 if (message.type === "offer") {
2   await pc.setRemoteDescription({
3     type: "offer",
4     sdp: message.sdp
5   })
6
7   const answer = await pc.createAnswer()
8   await pc.setLocalDescription(answer)
9
10  await signalingChannel.send({
11    type: "answer",
12    from: userId,
13    to: message.from,
14    sdp: answer.sdp
15  })
16 }
```

Listing 4 Receiving offer and creating answer

The `setRemoteDescription()` call processes the offer SDP received from the peer, informing the local `RTCPeerConnection` about the remote peer's media capabilities. The `createAnswer()` method then generates a compatible answer, which is set locally and returned via signaling.

4.4 ICE Candidate Exchange

As the peer connection is established, ICE candidates are gathered asynchronously and must be transmitted to the remote peer, as shown in Listing 5:

```
1 pc.onicecandidate = (event: RTCPeerConnectionIceEvent) => {
2   if (!event.candidate) return
3
4   await signalingChannel.send({
5     type: "candidate",
6     from: userId,
7     to: peerId,
8     candidate: {
9       candidate: event.candidate.candidate,
10      sdpMid: event.candidate.sdpMid,
11      sdpMLineIndex: event.candidate.sdpMLineIndex
12    }
13  })
14 }
```

Listing 5 Gathering and sending ICE candidates

The `onicecandidate` event fires when an ICE agent discovers a new candidate. Each candidate is serialized (containing the candidate string, media line index, and media ID) and sent to the peer. A null `event.candidate` signals completion of candidate gathering.

Candidates may arrive before the remote description is set. A candidate queue is maintained per peer to buffer candidates until the remote description is established. These queued candidates are then processed, as illustrated in Listing 6:

```
1 if (message.type === "candidate" && message.to === userId) {
2   const pc = pcMap.get(message.from)
3
4   if (pc.remoteDescription?.type) {
5     pc.addIceCandidate(message.candidate as RTCIceCandidateInit)
6   } else {
7     candidateQueue.get(message.from)?.push(message.candidate)
8   }
9 }
```

Listing 6 Receiving and queuing candidates

This ensures proper state management in the WebRTC connection establishment process, where candidates must be processed only after the remote description is available.

4.5 Receiving Remote Media Streams

When the remote peer's media arrives, the ontrack event is fired, as shown in Listing 7:

```
1 pc.ontrack = (event: RTCTrackEvent) => {
2   const remoteStream = event.streams[0]
3
4   // Attach stream to video element
5   videoElement.srcObject = remoteStream
6 }
```

Listing 7 Handling incoming remote streams

The ontrack event provides the incoming `MediaStream` containing remote peer's audio and video tracks. This stream is attached to a video element for playback, enabling participants to see and hear each other.

4.6 Media Control via Track State

Media can be toggled by controlling the `enabled` property of tracks without tearing down the connection, as demonstrated in Listing 8:

```
1 function muteAudio() {
2   pc.getSenders()
3     .filter(sender => sender.track?.kind === "audio")
4     .forEach(sender => {
5       if (sender.track) sender.track.enabled = false
6     })
7 }
8
9 function disableVideo() {
10  pc.getSenders()
11    .filter(sender => sender.track?.kind === "video")
12    .forEach(sender => {
13      if (sender.track) sender.track.enabled = false
14    })
15 }
```

Listing 8 Toggling audio and video

By setting `track.enabled = false`, the local peer stops sending audio or video while maintaining the ICE connection and `RTCPeerConnection` state. Remote peers receive this state change via RTCP.

4.7 Connection Cleanup

Proper cleanup prevents resource leaks when peers disconnect, as illustrated in Listing 9:

```
1 function closeConnection() {  
2   // Stop all local tracks  
3   pc.getSenders().forEach(sender => {  
4     sender.track?.stop()  
5   })  
6  
7   pc.close() // Close the peer connection  
8 }
```

Listing 9 Closing connections and stopping tracks

The `RTCPeerConnection.close()` method terminates the connection, stops ICE, DTLS, and SRTP processes, and releases all associated resources. Local tracks are explicitly stopped via `stop()` to release access to media devices, preventing them from remaining in use after the connection closes.

4.8 Group Conferencing with Full Mesh Topology

In the full mesh topology implemented in this application, group tutoring sessions establish multiple peer connections, as shown in Listing 10:

```
1 const pcMap = new Map<string, RTCPeerConnection>()  
2  
3 // For each participant, create and maintain a peer connection  
4 for (const peerId of remoteParticipants) {  
5   if (!pcMap.has(peerId)) {  
6     const pc = new RTCPeerConnection({ iceServers: ICE_SERVERS })  
7     pcMap.set(peerId, pc)  
8  
9     // Initialize peer connection as shown in Listings 1-2  
10  }  
11 }
```

Listing 10 Managing multiple peer connections

Each participant maintains $n - 1$ simultaneous `RTCPeerConnection` instances, where n is the total number of participants. Each connection independently handles offer/answer negotiation, ICE candidate exchange, and media streaming, enabling scalable multi-party communication.

4.9 User Interface and Meeting Room

Figure 10 shows the meeting room for conducting online tutoring sessions during an audio-video call between two participants (one tutor and one student).

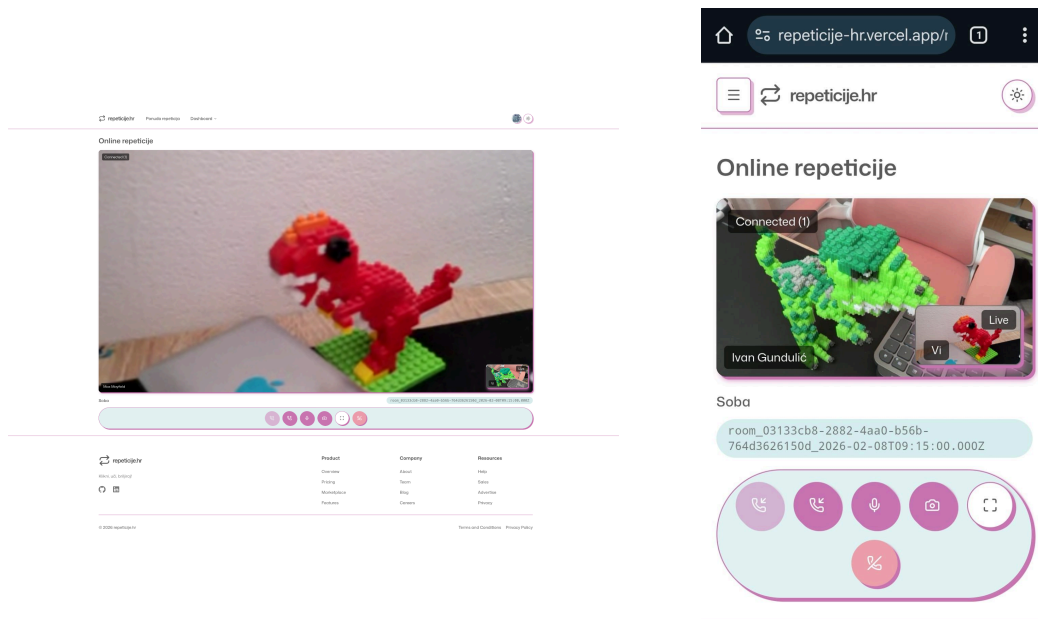


Figure 10 Meeting room during audio-video call between two participants: desktop, *macOS, Chrome* (left) and mobile, *Android, Chrome* (right)

Figures 11 and 12 show the meeting room during an audio-video call between multiple participants (one tutor and two students), illustrating the functionality of group online tutoring sessions across different browsers, operating systems, devices, and network conditions.

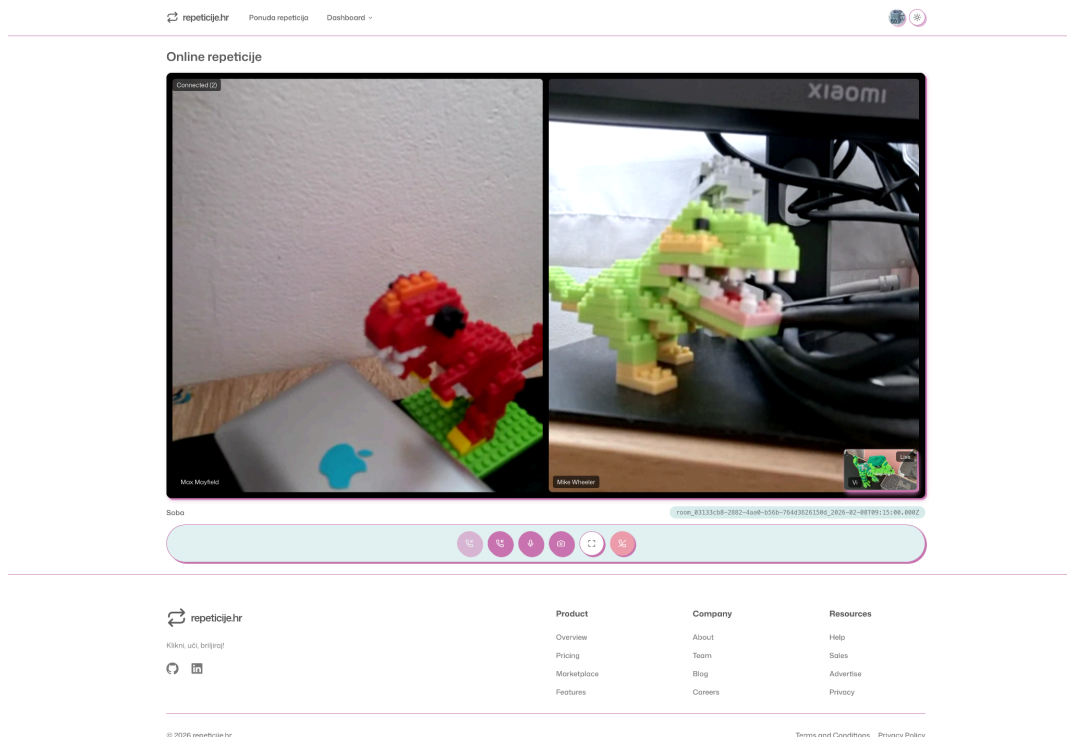


Figure 11 Meeting room during audio-video call between multiple participants: desktop, *macOS, Chrome*

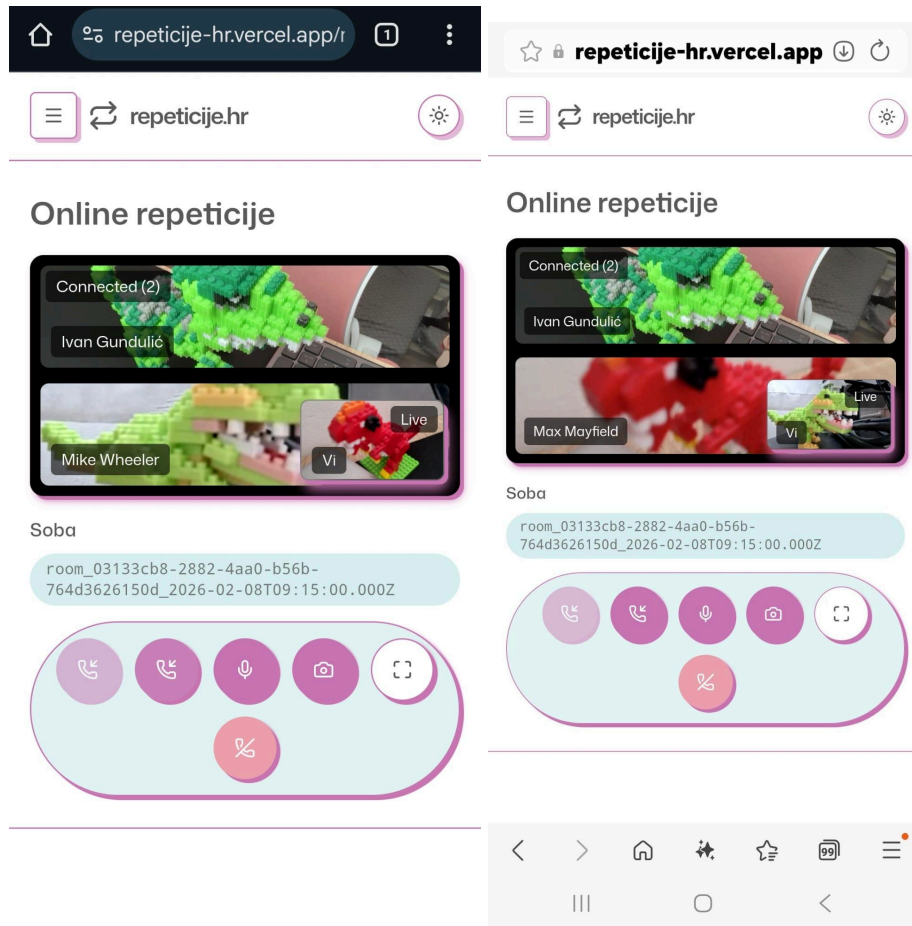


Figure 12 Meeting room during audio-video call between multiple participants: mobile, *Android, Chrome* (left), mobile, *Android, Samsung Internet* (right)

5. Conclusion

This paper presented WebRTC technology — a standardized FOSS collection of protocols enabling plugin-free peer-to-peer real-time audio, video, and data communication via modern browsers through signaling, ICE NAT traversal, DTLS-SRTP security, and RTP/SCTP transport. WebRTC was exemplified in practice through a full-stack Next.js application with Prisma ORM, PostgreSQL, and Supabase real-time signaling, serving as a student-tutor platform for online/onsite tutoring with fully meshed group video calls managed by the RoomCall React component. The implementation validated seamless cross-device functionality, secure connection phases, and scalable educational video conferencing without complex infrastructure. The successful integration demonstrates that modern web-based real-time communication no longer requires proprietary solutions — standardized, open WebRTC technologies provide robust, secure, and accessible peer-to-peer multimedia exchange for contemporary web applications.

REFERENCES

- [1] Alvestrand, H. (2021). Overview: Real-Time Protocols for Browser-Based Applications (WebRTC) (RFC 8825). <https://datatracker.ietf.org/doc/html/rfc8825>
- [2] Alvestrand, H. (2021). WebRTC Media Transport Independent Security Requirements (RFC 8830). <https://datatracker.ietf.org/doc/html/rfc8830>
- [3] Baugher, M., McGrew, D., Naslund, M., Carrara, E., & Norrman, K. (2004). The Secure Real-time Transport Protocol (SRTP) (RFC 3711). <https://datatracker.ietf.org/doc/html/rfc3711>

- [4] Handley, M., Jacobson, V., & Perkins, C. (2006). SDP: Session Description Protocol (RFC 4566). <https://datatracker.ietf.org/doc/html/rfc4566>
- [5] Mahy, R., Matthews, P., & Rosenberg, J. (2010). Traversal Using Relays around NAT (TURN) (RFC 5766). <https://datatracker.ietf.org/doc/html/rfc5766>
- [6] Papić, A. (2026). Razvoj web aplikacije za repeticije. Diplomski rad. Sveučilište u Splitu, Sveučilišni odjel za stručne studije, Split. <https://urn.nsk.hr/urn:nbn:hr:228:949522>
- [7] Rescorla, E., & Modadugu, N. (2012). Datagram Transport Layer Security Version 1.2 (RFC 6347). <https://datatracker.ietf.org/doc/html/rfc6347>
- [8] Rescorla, E., et al. (2022). The WebRTC Security Architecture (RFC 9143). <https://datatracker.ietf.org/doc/html/rfc9143>
- [9] Rosenberg, J., Mahy, R., & Wing, D. (2010). Interactive Connectivity Establishment (ICE) (RFC 5245). <https://datatracker.ietf.org/doc/html/rfc5245>
- [10] Rosenberg, J., Mahy, R., Matthews, P., & Wing, D. (2010). Session Traversal Utilities for NAT (STUN) (RFC 5389). <https://datatracker.ietf.org/doc/html/rfc5389>
- [11] Schulzrinne, H., Casner, S., Frederick, R., & Jacobson, V. (2003). RTP: A Transport Protocol for Real-Time Applications (RFC 3550). <https://datatracker.ietf.org/doc/html/rfc3550>
- [12] Stewart, R. (2007). Stream Control Transmission Protocol (RFC 4960). <https://datatracker.ietf.org/doc/html/rfc4960>
- [13] Uberti, J., Jennings, C., & Rescorla, E. (2024). JavaScript Session Establishment Protocol (JSEP) (RFC 9429). <https://datatracker.ietf.org/doc/html/rfc9429>
- [14] W3C. (2025). WebRTC: Real-Time Communication in Browsers (Recommendation). <https://www.w3.org/TR/webrtc/>
- [15] Westerlund, M., & Lundin, H. (2015). RTP Topologies (RFC 7667). <https://datatracker.ietf.org/doc/html/rfc7667>